

Introduction to UVM

Yasir Farooq

Prerequisites

You should have

- Fundamental knowledge of SystemVerilog
- Basic knowledge of OOP (Object-Oriented Programming)
- SystemVerilog based Layered Testbench

Objectives of the UVM based Verification

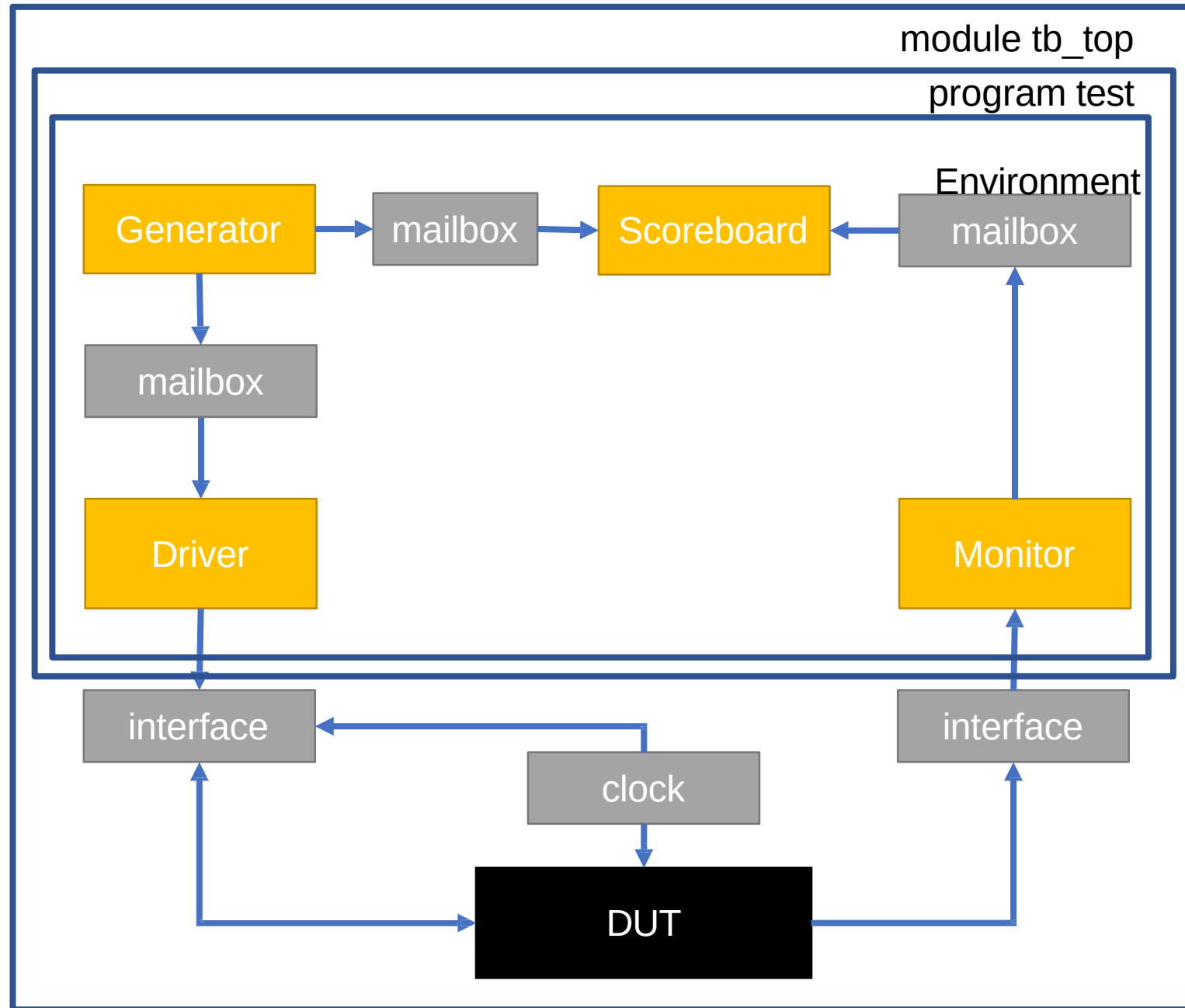
- Explore UVM Verification Component (UVC) architecture
- Create reusable UVCs
- Control stimulus generation
- Control environment behaviours
- Generate stimulus sequences and check responses
- Implement a scoreboard to check the behaviour of the design

What is UVM?

- The Universal Verification Methodology (UVM) has become the standard for verification of integrated circuits design.
- The UVM class library facilitates the implementation of testbenches.
- Each element of a UVM testbench is a component derived from an existing UVM class.

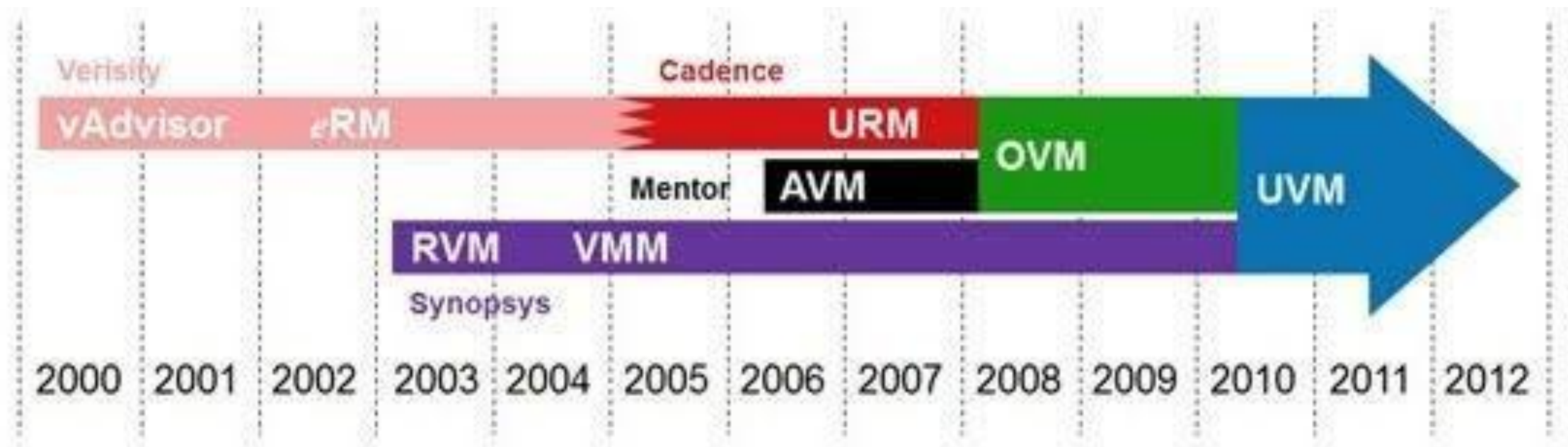
SV Layered Testbench

- Scalable and reusable for small systems **but**
- Not standard approach
- Increased Complexity
- Synchronization issues for System level simulations



UVM History

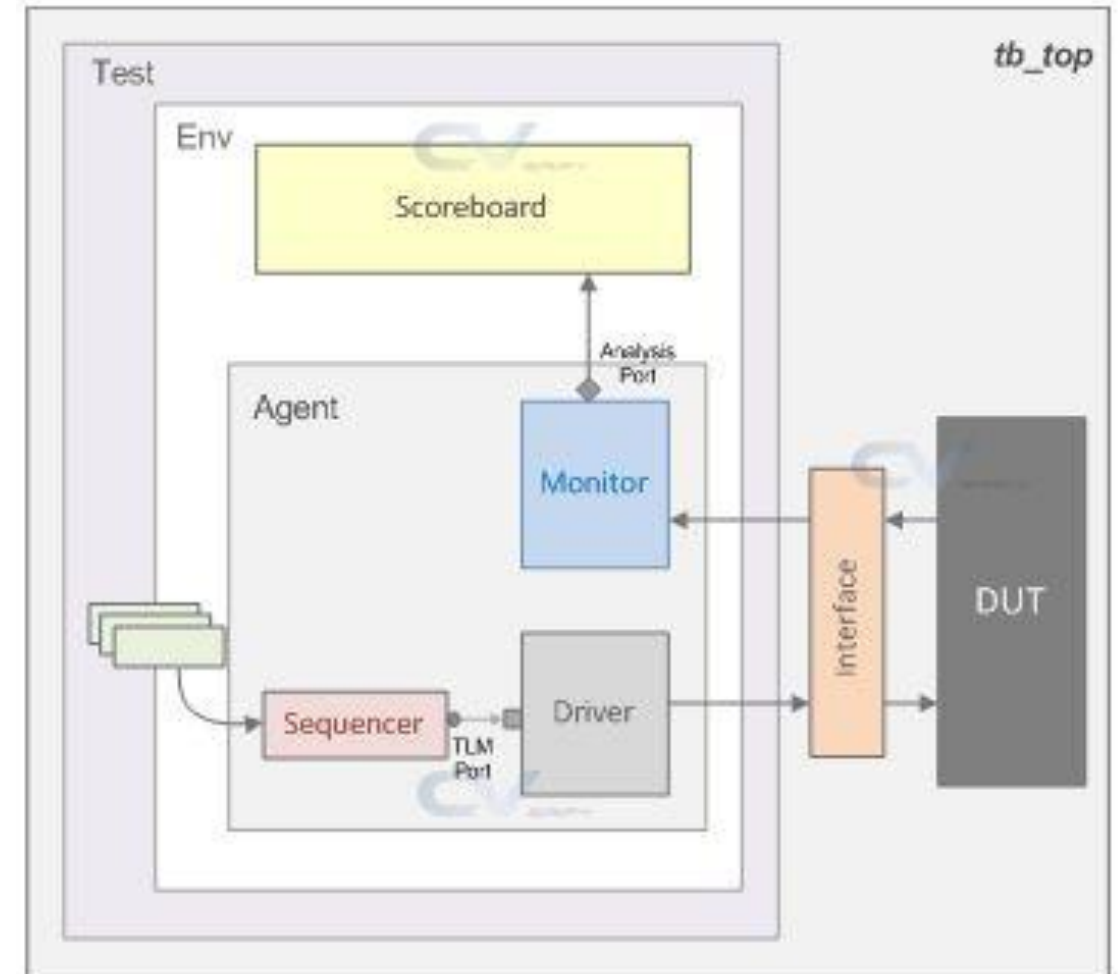
- UVM has advantages over other verification methodologies
- UVM is a standard



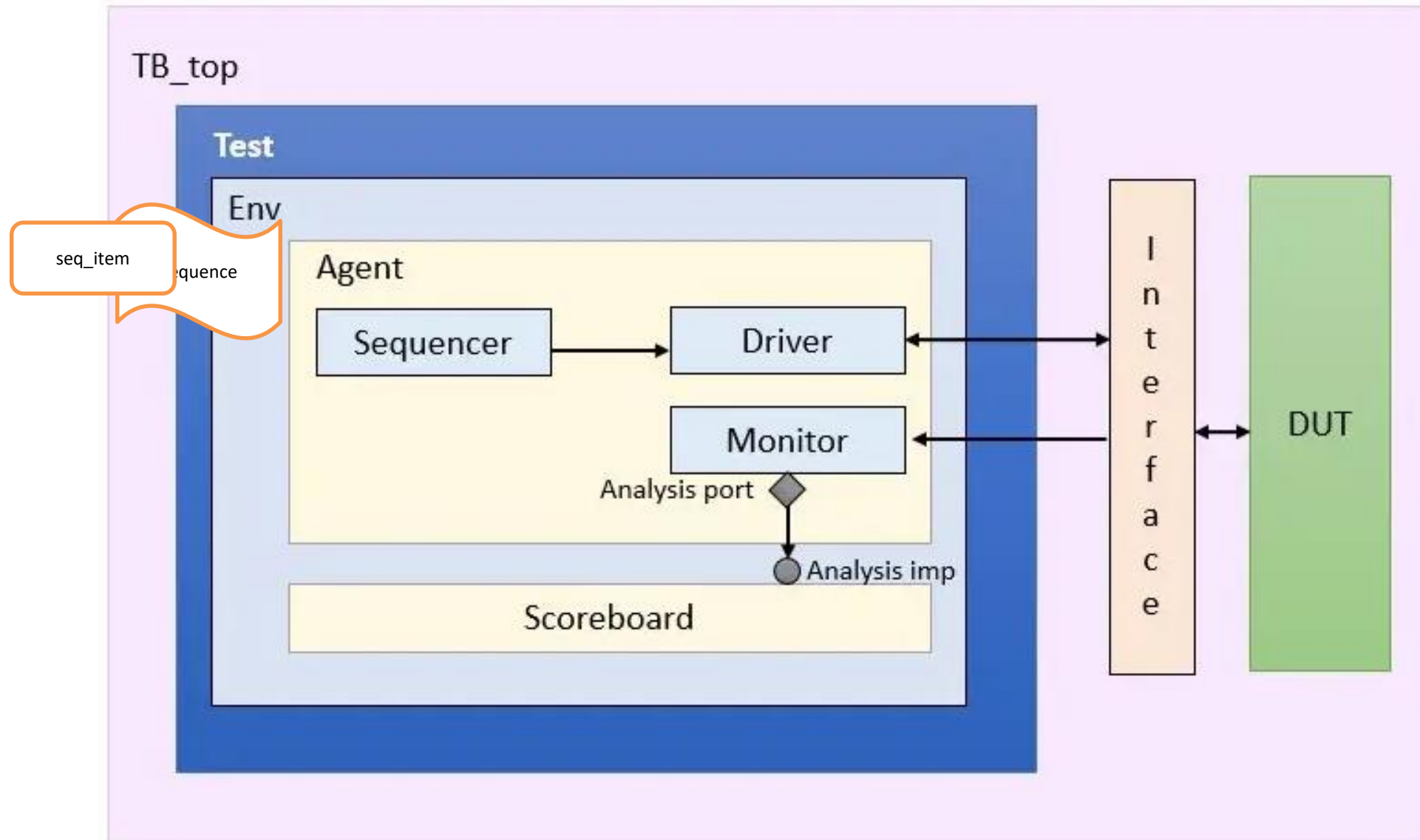
UVM Testbench Structure

Position of the UVM components in UVM testbench

- Testbench
 - DUT, UVM test
 - UVM sequence items, UVM Env
 - Agent, Scoreboard
 - Sequencer, Driver, Monitor



UVM Testbench Blockdiagram



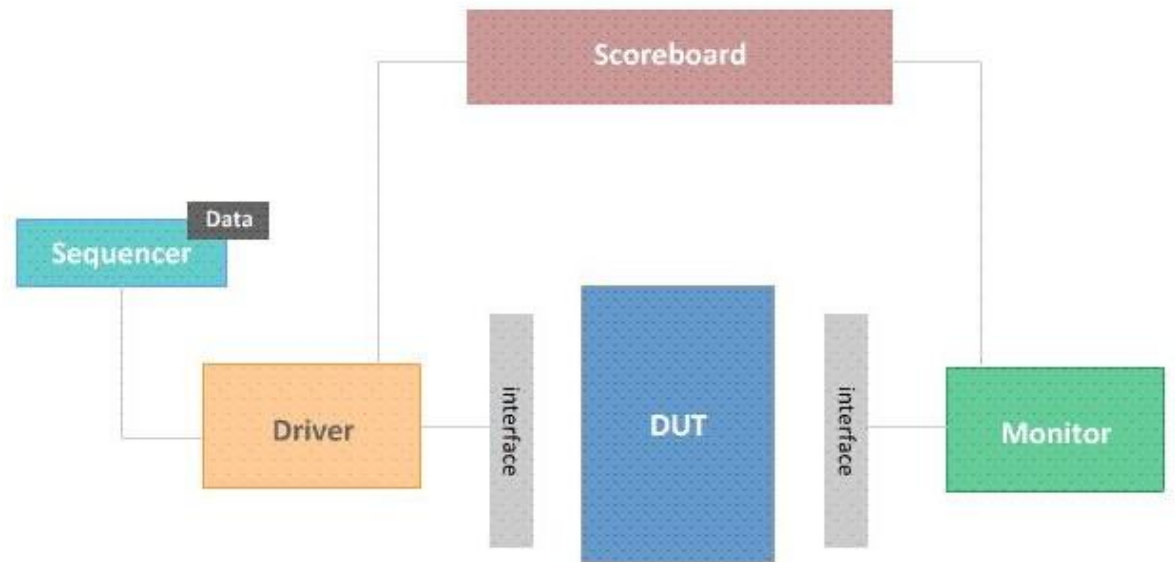
Layered Testbench vs UVM

- UVM has well defined foundation
- It has base classes for testbench components like driver, monitor, scoreboard etc
- UVM provides configurability
- Objects can be dynamically created and configured (concept of factory and configuration database)
- UVM testbenches are reusable and scalable
- Can be used for the unit level testing upto board level testing

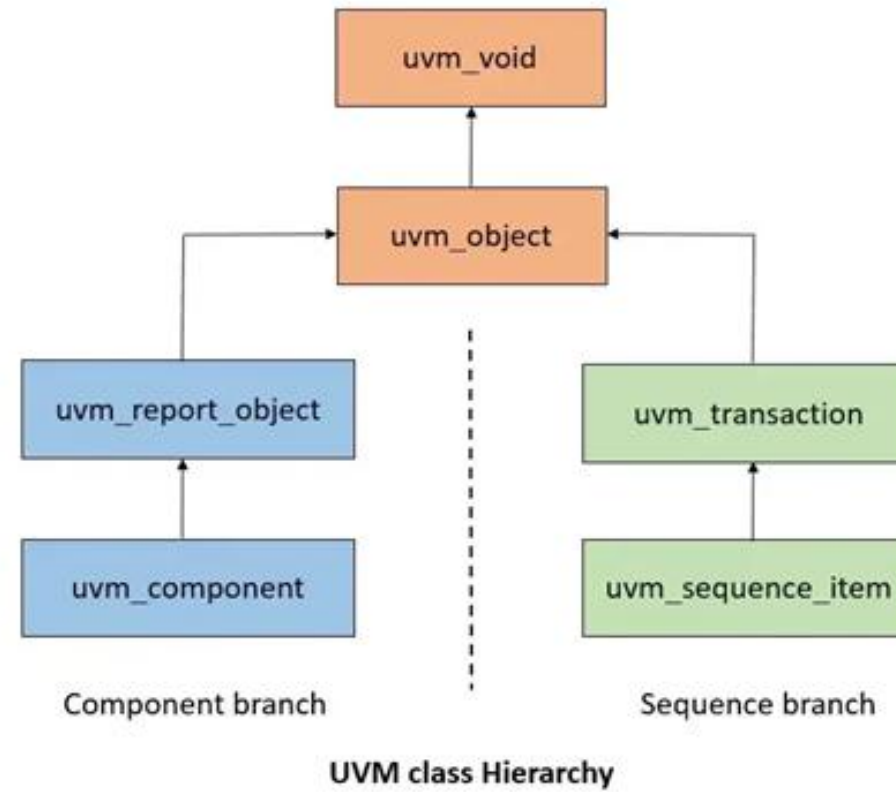
Layered Testbench vs UVM

- UVM provides automation, it already has components required for verification environment

System Verilog TestBench as we know it..



UVM Classes



Lab Exercise

Lab 1 UVM Environment Setup

To set up and test the UVM testbench